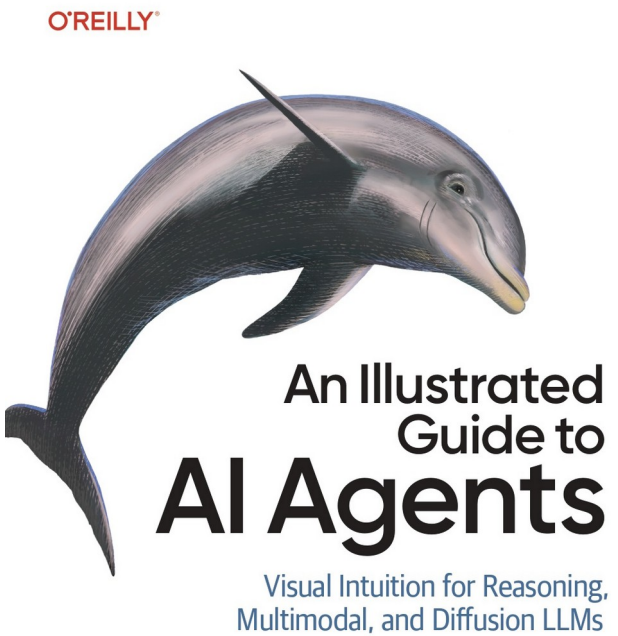
M3

Memory

Module 3 — Giving the agent a memory



307307 — BI Methods for Generative AI

Agentic AI · University of Petra

Lecturer: _____ Date: _____

Learning Objectives

1. Explain why LLMs are stateless and what that means for an agent.
2. Describe short-term memory as the conversation history fed back into the prompt.
3. Explain the context window limit, and manage it with trimming and summarization.
4. Describe long-term memory: how RAG works and what a vector database does.
5. Connect memory to context engineering and to reducing hallucinations.

The Problem — The Agent Forgets

In Module 2 the agent got a brain — but it is stateless.

Tell it your name, ask again, and it does not know. Every call starts from a blank slate.

Apps like ChatGPT feel like they remember only because they re-send the conversation each time. The model itself has no memory — we have to give it one.

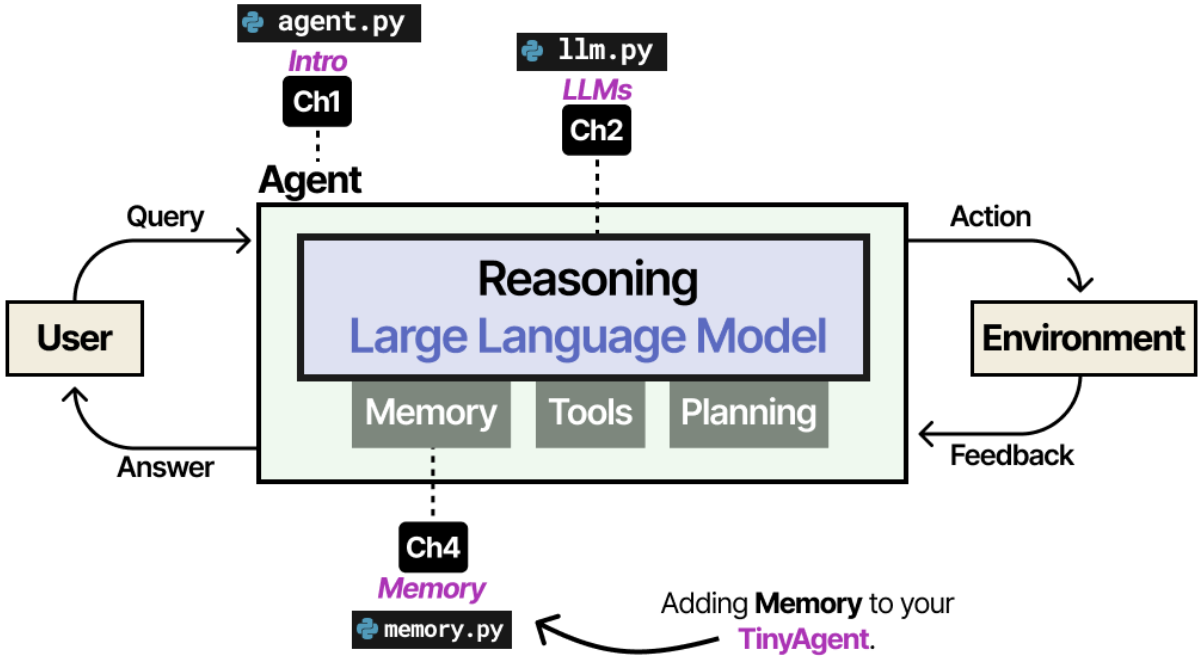


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026)

Types of Memory

Agent memory is modelled on human memory. The key split:

Short-term

Working memory
The ongoing conversation — data kept across calls.

For us: the chat history.

Long-term (external store, retrieved when needed)

Episodic	specific past events and actions the agent took
Semantic	general knowledge — your documents, a codebase, a wiki
Procedural	how to do things — often in the system prompt

Short-Term Memory = Conversation History

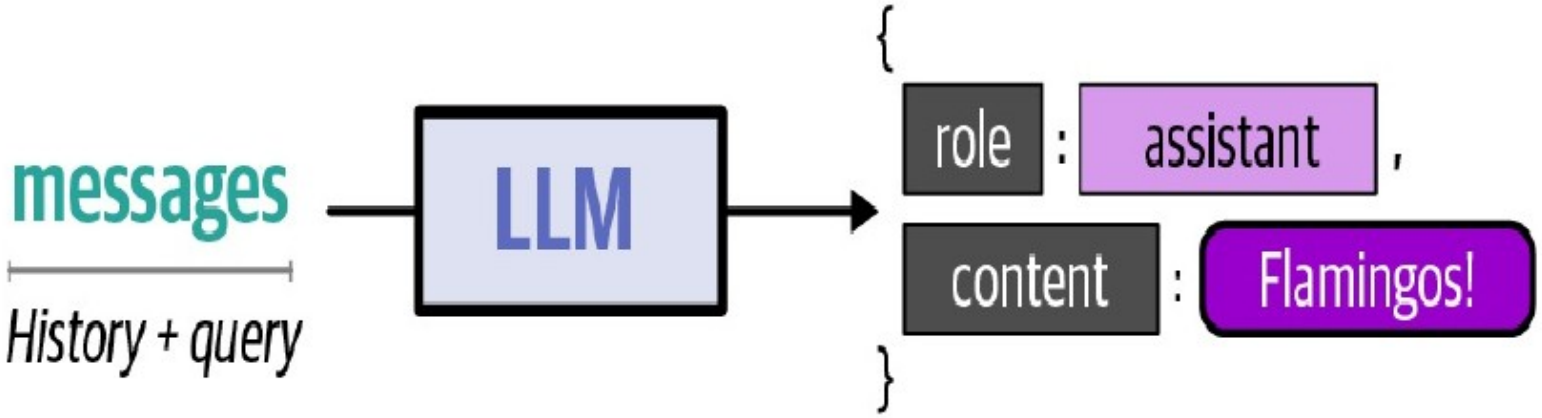


Figure 4-6. The LLM is given both the conversation history as well as the current query

Figure 4-6, An Illustrated Guide to AI Agents (O'Reilly, 2026)

The history is just a list of messages (role + content). We keep appending to it, and send the whole list back every turn.

The model does not truly remember — we tell it what the conversation was by putting it in the prompt.

The Catch — The Context Window

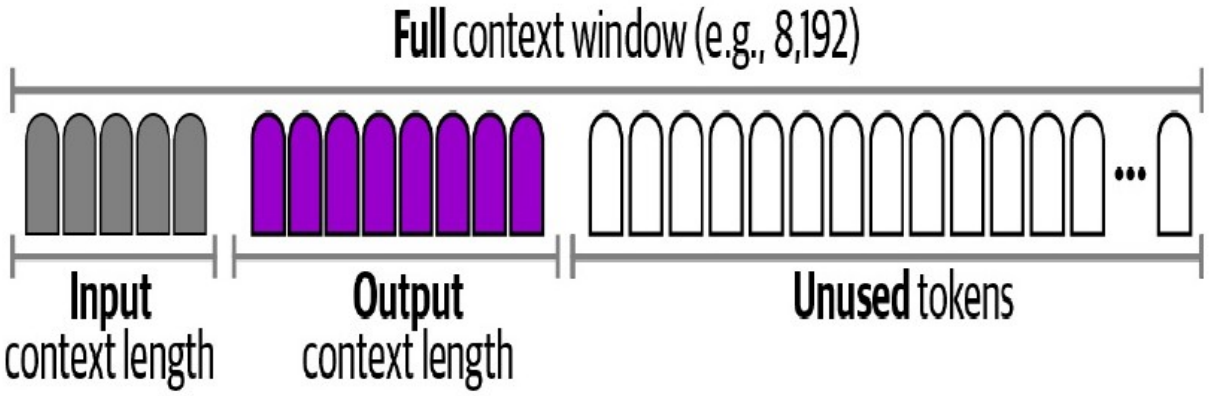


Figure 4-8. The full context window should manage both the input and output tokens

Figure 4-8, An Illustrated Guide to AI Agents (O'Reilly, 2026)

Limited tokens

The model can process only so many tokens at once — input and output together.

It overflows

As the history grows, it eventually will not fit: the answer is cut off, or nothing is produced.

Focus suffers

Too much context also makes it harder for the model to attend to what matters.

Keeping It Small — Trim or Summarize

Trimming

Keep only the most recent turns (e.g. the last two) and drop older ones. Simple — but you may lose something important from earlier.

Summarization

Use an LLM to compress older turns into a short running summary, kept alongside the recent messages. Saves space while keeping the gist — though over-summarizing can lose detail.

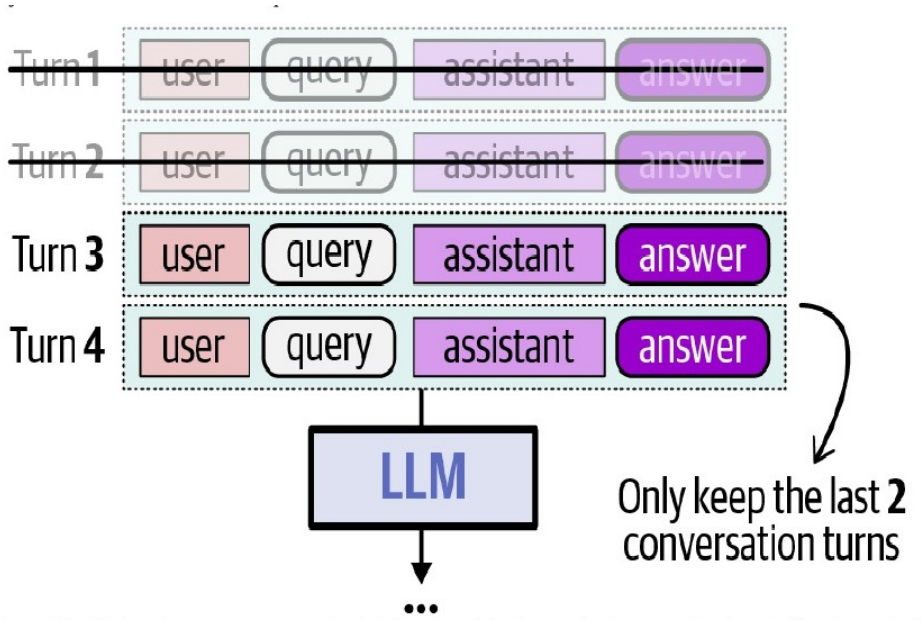


Figure 4-10. Efficient short-term memory may include keeping only the last couple of conversations instead of keeping track of the entire conversation history

Trimming — Figure 4-10 (O'Reilly, 2026)

Long-Term Memory — Retrieval (RAG)

When knowledge lives outside the conversation (your documents, a knowledge base), the agent retrieves it. This is RAG — Retrieval-Augmented Generation:

1. **Embed** — turn text into a numeric vector; similar meanings get similar vectors
2. **Retrieve** — find stored items most similar to the question (cosine similarity)
3. **Augment** — add the retrieved pieces to the prompt as context
4. **Generate** — the LLM answers using that added context

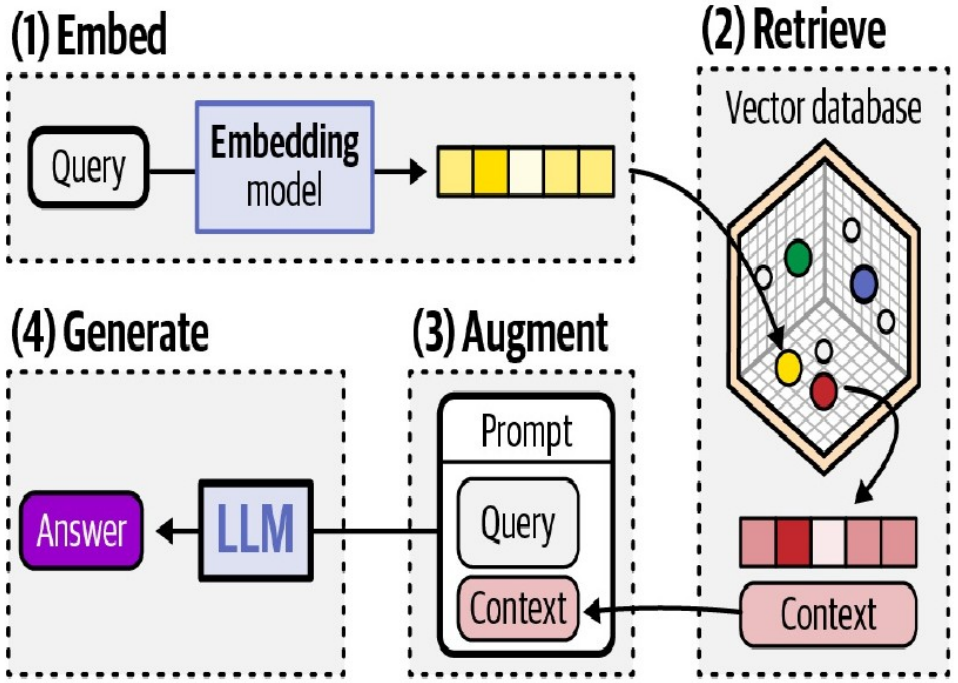


Figure 4-15. The full pipeline of retrieval- (2) augmented (3) generation (4)

Figure 4-15, An Illustrated Guide to AI Agents (O'Reilly, 2026)

The Vector Database — Where the Vectors Live

What it stores

Each vector together with its original text and, often, metadata such as the source or the date.

What it does

Returns the stored items whose vectors are most similar to a query vector — a nearest-neighbour search.

Why a database, not a plain list?

Comparing the question with every vector is fine for 8 documents (our lab) but slow for millions. A vector database builds an index that finds approximately the nearest vectors much faster. Examples: Chroma, Qdrant, Pinecone, Weaviate, Milvus, pgvector.

Making RAG Work — Three Things to Get Right

Chunking

Long documents are split into smaller chunks before embedding, so each vector stands for one focused piece of text.

Same embedding model

Embed the documents and the questions with the same model. Vectors from different models cannot be compared.

Retrieval can fail

A vector search returns the nearest items even when none of them is relevant. RAG adds facts to the prompt; it does not change the model — so weak retrieval gives a weak answer.

Why It Matters

Reduces hallucinations

Hallucination = confidently making things up. Handing the model relevant, trusted facts makes it far less likely to invent an answer.

Context engineering

Give the model the right information, not all of it. Memory is where this happens — choose what to keep, summarize, and retrieve.

A step further: agentic RAG

Instead of always retrieving once, the agent decides what to look up and how often — treating each knowledge source as a tool. We return to this idea in Module 5.

What You Will Build Today

1 See the problem

Run a stateless agent and watch it forget your name.

2 Memory module

A ~15-line class: add() and get_messages().

3 Remember

Wire Memory into the TinyAgent — it holds a conversation.

4 Trim

Use TrimmingMemory to keep only the last few turns.

5 Retrieve (a tiny RAG)

Embed a few documents, find the ones closest to a question with cosine similarity, and let the LLM answer from them.

A run-and-observe lab: run each cell and read the output. One small exercise at the end.

Key Takeaways

1. LLMs are stateless; a memory module re-sends the information each call.
2. Short-term memory is the conversation history — a list of messages.
3. The context window is limited → manage it with trimming or summarization.
4. Long-term memory = external knowledge via RAG (embed → retrieve → augment → generate); the vectors are kept in a vector database.
5. It is all context engineering, and good memory reduces hallucinations.

Discussion

Q1 Why is an LLM 'stateless', and how does a memory module make it seem to remember?

Q2 Name two ways to keep a long conversation within the context window, and a downside of each.

Q3 List the four steps of RAG. How does retrieved context help reduce hallucinations?

Q4 What does a vector database store, and why must documents and questions be embedded with the same model?

Next: the Tools module — so the agent can finally act on the world.